

HD-Code: End-to-End High Density Code for DNA Storage

Jianjun Wu¹, Shufang Zhang¹, *Member, IEEE*, Tao Zhang¹, and Yuhong Liu², *Senior Member, IEEE*

Abstract—With the rapid development of digital information techniques, the use of DNA media for information storage is considered as the future direction of data storage. Existing DNA storage schemes simply map compressed binary multimedia data into DNA base data, which has the disadvantages of data loss, low logical storage density and high cost of synthesis. This paper presents an end-to-end high density DNA encoding algorithm (referred to as HD-code, where HD stands for high density). The novelty and contributions of this work contain three parts. First, by taking full advantage of the statistical characteristics of the original multimedia data and considering the biological constraints on the DNA bases, the proposed scheme achieves higher logical storage density and improves the flexibility and consistency in data storage. Second, by performing data conversion, the proposed scheme can effectively encode extreme images with large proportion of single color. Third, the proposed method can reconstruct high quality images and reduce synthesis costs by yielding better rate-PSNR (Peak Signal to Noise Ratio).

Index Terms—DNA storage, high density code, DNA encode, end-to-end.

I. INTRODUCTION

IN RECENT years, the exponential growth of data volume has put great demand on storage media [1]. Traditional digital storage devices, such as disks, CDs, U disks and HDDs [2] (hard disk drives), have certain advantages in terms of retention time (i.e. data is still recoverable), access speed (i.e. latency and bandwidth of data access), and energy cost of data. However, its storage capacity growth rate is much lower than the growth rate of digital information, which cannot meet the requirements of today's huge data volume. DNA molecule is considered as the next generation information storage medium [3] because of its double helix structure and base stacking interaction.

DNA data storage consists of three parts: base sequence conversion, biosynthesis and storage. Base sequence conversion converts existing binary data into base sequences

by compression encoding. With efficient DNA compression encoding, the number of base sequences can be significantly reduced, which will further reduce the cost of DNA synthesis effectively. Therefore, compression encoding is the most critical step in the overall DNA storage method. In IEEE TRANSACTIONS ON NANOBIOSCIENCE journal, B. Cao *et al.* used Minimum free energy code for DNA storage [4]. The effectiveness of code determines the overall storage effect. As a result, many recent studies have been proposed to perform efficient DNA encoding. In [5], a DNA data storage scheme with variable length oligos is proposed, in which a hybrid DNA mapping is introduced to convert binary digital data into DNA records. Experimental results show that the net information density (i.e. the number of bits per nucleotide) of 1.67 bits/nt is achieved with 91% information capacity. In [6], a coding method using binary mapped DNA sequence for DNA data storage is proposed, and the coding method has been proved to be efficient with a coding rate of 1.917 bits and low complexity. In [7], In addition to using A, T, C, G, the method also uses the coding symbols of degenerate bases. By increasing the DNA sequence, the amount of DNA synthesis per storage unit is reduced. Experiments show that the proposed method achieves 3.37 bits/nt information capacity. In [8], a variable long-run limit code (VL-RLL) and an optimized low-density parity-check code (LDPC) are presented to convert binary data into DNA encoding, and an improved LDPC code is optimized using an information transfer algorithm. Experimental results show that the storage density of the hybrid encoding method is 1.98 bits (bits/nt). A DNA storage scheme for DNA fountain code has been proposed [9]. Experimental results show that the proposed method can store binary multimedia data as DNA base data completely, and the scheme is robust, feasible and has a high storage capacity for information. In [10], a CRISPR-Cas adaptive system method is proposed, which encodes the binary data of black-and-white image and a short piece of pixel value into DNA base data, and stores the genome of living bacteria. Experiments show that this method can store real data stably. Paper [11] presents a Huffman-like DNA compression encoding that effectively passes binary streams through DNA-Huffman encoding.

The existing base sequence compression encoding methods are mainly designed to process the compressed binary multimedia data, which are obtained after applying traditional multimedia compression encoding on the original multimedia

Manuscript received February 24, 2021; revised May 28, 2021; accepted July 7, 2021. Date of publication August 3, 2021; date of current version October 1, 2021. (Corresponding author: Shufang Zhang.)

Jianjun Wu, Shufang Zhang, and Tao Zhang are with the School of Electrical and Information Engineering, Tianjin University, Tianjin 300072, China (e-mail: june-woo@foxmail.com; shufangzhang@tju.edu.cn; zhangtao@tju.edu.cn).

Yuhong Liu is with the Department of Computer Science and Engineering, Santa Clara University, Santa Clara, CA 95053 USA (e-mail: yhliu@scu.edu).

Digital Object Identifier 10.1109/TNB.2021.3102122

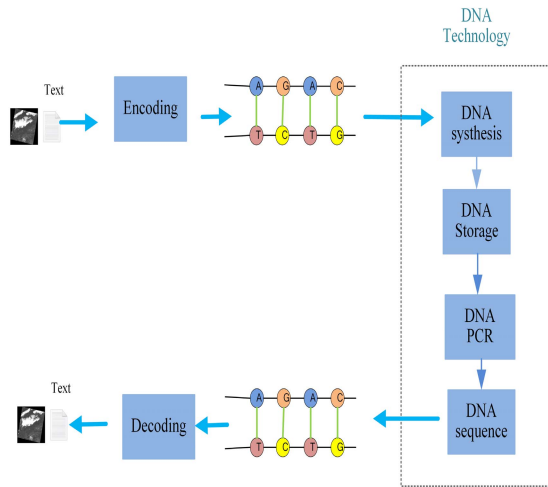


Fig. 1. DNA storage overview.

data. As a result, the impact of these existing studies highly relies on the results of the adopted multimedia compression encoding, and is therefore often limited. In this paper, we aim to address the above issues with the following contributions. First, an **end-to-end High Density DNA code** (HD-code, where H stands for high and D stands for density) is proposed, which **takes full advantage of the statistical characteristics of the original multimedia data, and considers the biological constraints on the DNA bases**. Experimental results validate that the proposed method increases the flexibility and consistency of the overall DNA storage, and achieves higher logical storage density than existing storage schemes. Second, by performing data conversion, the proposed scheme solves the problem where proportion of single pixel is too large to be encoded. Therefore, extreme images with large proportion of single color can be encoded effectively. Third, the proposed method yields better rate-PSNR by storing large amount of pixel data in small number of nucleotides. It indicates that the proposed scheme can reconstruct high quality image and have low cost of synthesis. In addition, this paper adopts an error correction algorithm based on read error correction [12], which can effectively correct errors in the process of DNA sequence synthesis, sequencing and storage and ensure the reliability of data storage.

The rest of this paper is arranged as follows. The second section introduces the DNA storage system; the third section describes the algorithms proposed in this paper, including the conversion of image data, fast sorting of data probability, DNA base sequence compression coding based on quadtree theory, and error correction algorithm based on read error correction. The fourth section is the results and analysis of the experiment part, followed by the summary of this paper.

II. DNA STORAGE OVERVIEW

The entire DNA storage system overview is shown in Figure 1. The steps for storing DNA base data in biosynthesis include DNA synthesis, storage, Polymerase chain reaction(PCR) amplification, and sequencing. When decoding source data, prior nucleotide extraction is required after a

biological process. DNA data decoding is the reverse process of DNA encoding, which recovers the source data information. Nucleotide extraction is performed by scanning the entire DNA strand vector.

A. DNA Encoding

DNA compression encoding is the key component of the DNA storage system. It mainly compresses the original sources (e.g. image and text data) into DNA base data; and when the reverse process is performed, the original source data can be retrieved by decoding the DNA base data. Compression encoding is equivalent to writing data to the DNA storage system. The purpose of compression encoding is to convert binary data into DNA base information sequences after algorithmic processing. With the DNA base information sequence, subsequent DNA biosynthesis can proceed. The reverse process of encoding when decoding digital data mainly extracts DNA base sequences from DNA storage systems and converts them into raw digital information.

B. DNA Synthesis

All single-stranded DNA is chemically synthesized by a single nucleotide. The coupling efficiency resulting from DNA synthesis is the probability that a nucleotide binds to a related single strand. The coupling efficiency of each synthesis step is higher than 99%, and the error of coupling efficiency decreases with the increase of single chain length. But the size of oligonucleotides is limited, and about 200 nucleotides can be synthesized at most [13]. In actual DNA synthesis, a large number of starting sites are required, and many short DNA chain by-products are also produced. Although DNA synthesis is prone to errors, a variety of modern technologies are developing rapidly for data repair, including nucleotide excision repair(NER) and base excision repair(BER). Nucleotide excision repair is to repair the nucleotide excision and repair of the whole genome through damage recognition. NER mainly repairs DNA damage that affects the regional chromosome structure. DNA damage includes the pyrimidine dimer caused by ultraviolet rays, the bond between chemical molecules or proteins and DNA, and so on. BER is mainly used to repair the damage that has a small range and has little effect on the structure of DNA double-stranded helix. The process of BER first removes damaged bases by DNA glycosylase. Then, the phosphodiester bond is cleaved. The missing bases are synthesized by DNA polymerase to complete the BER process.

C. DNA Storage

DNA storage converts the source text and image data to DNA bases AGCT, a process in which digital signals are converted to chemical signals. Usually an 8-bit data is represented by a base that must be less than 8 to achieve compression. DNA is so dense that a kilogram of DNA can store all the data in the world, which can save a lot of space and power consumption.

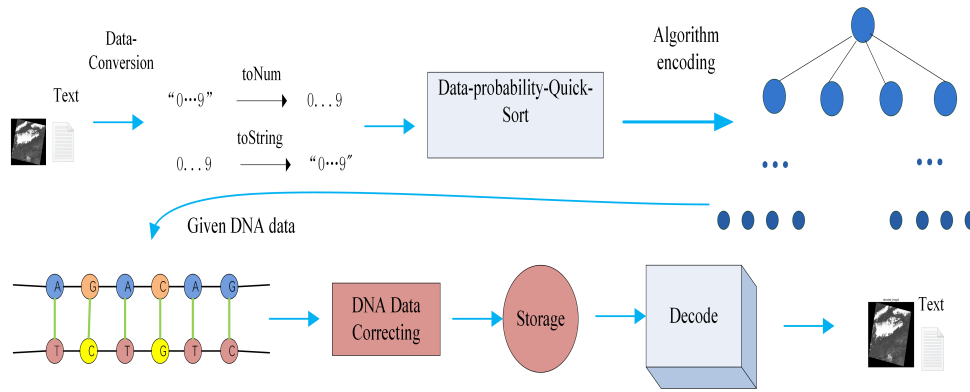


Fig. 2. Encoding Framework: A given image or text file as the source data is read directly by an algorithm. If the image data is read, it is first converted from character to integer, then the probabilities of each data type are sorted quickly, and then the algorithm encoding of the data is performed. The algorithm calculates the type of data, and then calculates the probability of occurrence of each data. After calculating the probability, each data is sorted in descending order according to the probability value. The data is quadrupled according to probability, and quadtree nodes are generated continuously. Finally, DNA base data is generated. DNA base data is stored biologically. When decoding, the primers need to be extracted first, then the source data can be extracted by the decoding algorithm.

D. DNA PCR

PCR is a fast way to copy DNA samples. Many PCR methods require a thermal cycle through which the DNA strand can gently depend on the reaction to achieve DNA replication. PCR first separates the two strands of DNA double helix at high temperature after nucleic acid deformation. The temperature was then lowered to allow the primers to fuse with DNA complementary sequences. Two DNA chains serve as templates for the polymerase and are replicated using enzymes. This process proceeds continuously by PCR, reaching the DNA itself as a replication template and exponentially amplifying the DNA template.

E. DNA Sequence

DNA sequencing is a technique for processing the sequence of nucleotides in DNA. Sequencing can quickly determine the four base sequences. Large-scale DNA sequencing can be used to sequence very long segments of DNA or to generate many short sequences. To improve the efficiency of large-scale sequencing, a manual gel is needed. Many sequencing techniques require amplification of a single DNA strand in vitro because sequencing is not susceptible to single-molecule sequencing.

III. METHODS

The compression encoding efficiency of existing DNA storage schemes is still low, and the cost of DNA encoding cannot be reduced because of the high cost of biosynthetics. This paper presents a DNA data storage method based on quadtree partitioning encoding, which can convert image and text data directly into DNA base sequence data for storage. The innovation is that without converting image and text data into binary data before compression encoding, it can be directly encoded into DNA base data by quadtree encoding. This algorithm can make the biosynthetic technology of the overall DNA storage scheme more operable, and increase the flexibility and compatibility of the overall DNA storage scheme. Its encoding framework is shown in Figure 2.

Image and text data contain a large number of statistical characteristics. If the probability characteristics of the data are well defined, it will help efficient encoding and compression [14]. Since the distribution of data in image and text files is not uniform, compression using the probability of data occurrence is an effective scheme. Furthermore, each data value in the image data contains three primary colors, red, green and blue, so the image data occupies a large repetition ratio in the overall data collection. To take advantage of these characteristics, we propose to first scan the image data using a statistical method and calculate the probability of each metadata for subsequent processing.

The probability distribution of the data then serves as the basis to encode the input data into an ATCG quaternion, which is the most important component for the proposed DNA storage system. DNA quadtree coding can effectively reduce the code length by assigning fewer bits to data with higher probability and more bits to data with lower probability. Specifically, during DNA encoding, the proposed algorithm generates a quadtree based on the probability of the occurrence of each character in the source data. Characters with the highest occurrence probability of source data characters will be encoded at the top of the quadtree and characters with lower occurrence probability will be placed at the bottom of the quadtree. After quadtree construction, 0-A, 1-G, 2-C, 3-T are used to map to DNA bases. Biological processes for DNA base storage after AGCT data are obtained. Quaternary numeric information is obtained by comparing the base data of the entire DNA chain vector with the map. For the decoding process, quaternary digital information is decoded one by one using DNA quadtree decoding algorithm to obtain the original data.

A. Data Conversion of Small Probability Values

In conventional image compression, some image data with very small occurrence probabilities are often overlooked by the encoding process, which negatively influences the effect of image coding compression. Due to the wide range of data

values in a normal image (i.e. 0 to 255), there is a high probability that one or some data values will appear much less frequently than other data values. In this paper, in order to solve the loss of pixel value in image coding compression due to the small probability values, an integer type of pixel value data is converted to character type data, so that the range of pixel values is between 0 and 9. As a result, the small probability pixel value is balanced with the high probability pixel value.

In the process of image processing, the pixel data of an image is read into the text file in advance. Specifically, when reading from a text file, the numerical values can be read either as an integer number, or as multiple characters. For example, for an original pixel data 205, it can be read as an integer value “205”, or as three characters ‘2’, ‘0’, and ‘5’. In an extreme case, if the values of all the pixels in the image are 205, reading the pixel data as an integer “205” will lead to a 100% frequency, which makes it very challenging for encoding. However, if the pixel data is read as three characters ‘2’, ‘0’ and ‘5’, the frequency of each character will be much more evenly distributed, as “33%”, “33%” and “33%”, making the later encoding much easier. This kind of data conversion reduces the problem of a single pixel’s excessively large ratio.

Character set $\sum \subseteq N$ (Characters can be mapped to numbers in computer) can be known in computer technology [15]. Since character set is a finite subset of natural number set, it can be considered that character coding in computer is to encode numerical value (for example, in ASCII code, character ‘0’ can be encoded into integer 0). Therefore, the set of characters and integers in the computer can form a mapping relationship. For simplicity of application, it can be assumed that the character ‘0’ is mapped to the number 0, and so on. For a normal character c , we use $||c||$ to represent integer.

We extract two types of applicable constraints from [15], i.e., integer and character conversion constraints:

(1) The constraint of an integer ϕ_i is a linear constraint for all integer variables in Z and for all $x \in X$, and for $X \rightarrow N$ is a string length function.

(2) For a number string conversion constraint, $x = toStr(n)$ is the same as $n = toNum(x)$. The general form of ϕ_s is $n = toNum(x)$.

In addition to the above two conversion rules. For the character variable x , the field limit is set to $\mathcal{R}(x)$. $\mathcal{R}(x)$ covers all possible values.

Figure 3 shows an example of the proposed data conversion. Please note that the proposed data conversion operations are performed only on extreme images, where the occurrence probabilities of certain image data values are very small, but not on text files with uniform data types.

When processing image data, there are some extreme cases, in which the probability of a pixel value (ranging from 0 to 255) is dominant and cause other pixel values to be overlooked for subsequent segmentation encoding. Such cases can be well addressed by the proposed data type conversion, which, by considering only ten data types (i.e. 0 - 9), can homogenize the probability of each data type, and perform normal codec. Figures 4 and 5 show the data probability distributions for extreme images before and after data conversion,

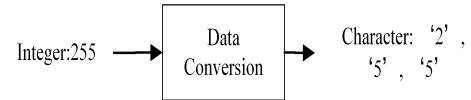


Fig. 3. Data conversion example.

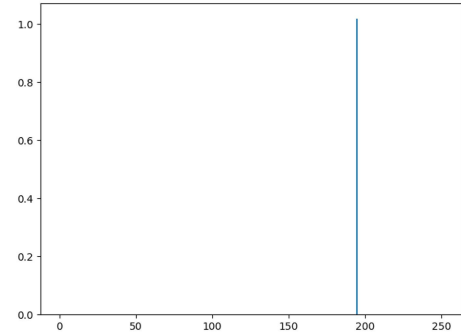


Fig. 4. Unconverted pixel probability distribution.

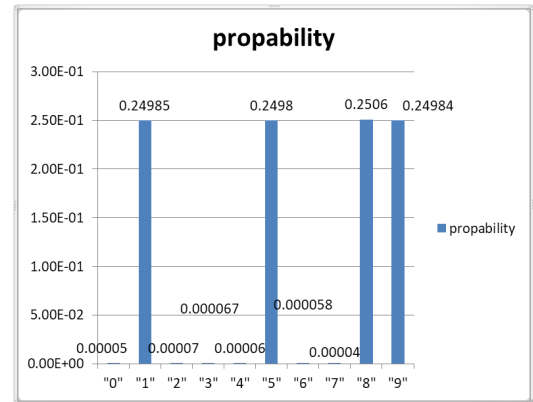


Fig. 5. Probability distribution after data conversion.

respectively. It is clear that when no data processing is done, there is a pixel data with a probability of approaching 1 at an approximate location of 200. This results in the inability to split. However, after the proposed data conversion, several probability values of the data can be counted for encoding.

B. Quick Sorting of Image Data Probability

Combining random data probabilities into ordered data combinations makes it easy to locate the size probabilities of the data types for encoding. Therefore, in this paper, we propose to sort image and text data based on their probabilistic ranking in descending order. Among different sorting algorithms, we adopt the quick sort algorithm because at the same data scale, the quick sort algorithm is much more efficient than other commonly used sorting algorithms. As the amount of compressed encoding data increases, the amount of data that needs to be sorted increases.

The time complexity of quick sort is $O(n \log n)$, reducing the time complexity for subsequent DNA compression encoding. We can analyze the time complexity of quick sort [16], [17].

We assume that the number of comparisons T_n . The time of the number of comparisons is our time complexity.

We give the definition of T_n .

$$T_n = (n+1) + \left(\frac{T_0 + T_{n-1}}{n}\right) + \left(\frac{T_1 + T_{n-2}}{n}\right) + \cdots + \left(\frac{T_{n-1} + T_0}{n}\right) \quad (1)$$

We replace n with $n-1$, and then subtract the difference:

$$\begin{aligned} \frac{T_n}{n+1} &= \frac{T_{n-1}}{n} + \frac{2}{n+1} \\ &= \frac{T_{n-2}}{n-1} + \frac{2}{n} + \frac{2}{n+1} \\ &= \frac{2}{3} + \frac{2}{4} + \cdots + \frac{2}{n+1} \end{aligned} \quad (2)$$

We can approximate the time complexity by transforming the integral.

$$T_n \approx 2n \ln n \approx O(n \log n) \quad (3)$$

The main idea is to divide the probability set to be sorted into two independent parts, where one part of the probability set to be sorted is smaller than the other part of the probability set, and then apply the quick sorting method to sort the two parts separately. By performing the sorting in a recursive way, the whole probability set will be sorted in descending order at the end [16], [17]. Below is a description of the algorithm for probabilistic ranking.

- a) Set two variables i and j at the beginning of sorting: $i = p - 1, j = r - 1$;
- b) With the r -th data as the key data, it is assigned to key, i.e., $key = A[r]$;
- c) Search forward from j , that is, search forward from back to j , find the first value $A[j]$ larger than key, and exchange the values of $A[j]$ and $A[i]$;
- d) Search backward from i , that is, search backward from front ($i++$), find the first $A[i]$ less than key, and exchange the values of $A[i]$ and $A[j]$;
- e) Repeat steps c and d until $i = j$;

C. New Quadtree DNA Encoding

In this paper, on the basis of the occurrence probabilities of image or text data, quadtree coding is performed to convert the source data into DNA base sequences. Tree data structure can compress, store, and retrieve data. Data structures commonly used in computer science include binary and quadruple trees. The compression rate and decoding speed of the data stored in the tree are all related to some parameters of the tree: weighted path length, tree height, node number, etc. The quadtree we define in this work has at most four sub-trees at any given vertex. When each node of a quadtree has four sub-trees, it is a full quadtree. Quadruple tree makes the data with high probabilities at the top of the tree and the data with small probabilities at the bottom of the tree, so as to achieve data compression.

Quadruple tree has advantages over binary tree coding due to the following reasons. Specifically, the minimum height of quadruple tree is smaller than that of binary tree, and

Algorithm 1 Probability Sorting

Require: A-array, p, r

```

1: function Partition(A, p, r)
2:    $x = A[r]$ 
3:    $i = p - 1$ 
4:   for  $j = p \rightarrow r - 1$  do
5:     if  $A[j] \leq x$  then
6:        $i = i + 1$ 
7:       exchange  $A[i]$  and  $A[j]$ 
8:     end if
9:   end for
10:  exchange  $A[i + 1]$  and  $A[r]$ 
11:  return  $i + 1$ 
12: end function
13:
14: function PSort(A, p, r)
15:   if  $p < r$  then
16:      $t = \text{Partition}(A, p, r)$ 
17:     PSort(A, p, t-1)
18:     PSort(A, t+1, r)
19:   end if
20: end function

```

the minimum and maximum height of quadruple tree is one third of that of binary tree when node n tends to approximate infinity. In the case of a k -tree [18], the minimum height of the tree is $\log_k n$. Hence, we can calculate $h_q = \log_4 n = \frac{1}{2} \log_2 n = \frac{1}{2} h_b$, where h_q is the height of the quadruple tree; and h_b is the height of the binary tree. Quadruple trees have better coding performance than binary trees because of their superiority in height, as shown below.

The related equations of binary tree are derived according to [18]. Suppose that there are n nodes in the binary tree, and each node has two subnode. A node is attached to a free leaf (Definition: free leaf is a node without any children) and a subtree (Definition: subtree has child nodes). Let m represent a node with only free leaves. $n = m + 2$ and $h_b = m + 1$, we can derive

$$h_b = n - 1 \quad (4)$$

The related equations of quadtree are analyzed in [18]. Suppose that there are n nodes in the quadtree, one of which is a dependent subtree node, and the other three are free leaf nodes. There are r ($1 \leq r \leq 3$) free nodes in the first layer (Definition: the first layer is the layer below the root node.) and three free nodes in the all middle layer. (Definition: the middle layer is the layer except for the last layer and the first layer.) Suppose m is the number of free nodes in the middle layer. $n = r + 3m + 4$, $1 \leq r \leq 3$ and $h_q = 1 + m + 1$, we can derive that

$$h_q = 1 + \frac{n-r-2}{3}, \quad 1 \leq r \leq 3 \quad (5)$$

Quadtree coding method is based on statistical model design, which sorts the probabilities of the input data in descending order and encodes the data accordingly. The statistical data set $D = d_1 \dots d_m$ represents all the data values

in a text or image file. The quadtree encodes each data in the set as a node in the tree.

The quadtree algorithm encodes data by constructing a code tree. In particular, a quadtree has left, left-mid, right, and right-mid nodes that can be affixed with A, G, C, and T bases, respectively. The codeword encoded for data d in data set D is the code along the path from the leaf data d to the root. Quadtree coding algorithm constructs a quadtree based on the occurrence probabilities of data D . All nodes of the quadtree have a series of probability weights $p_1 \dots p_n$, and the quadruple tree can achieve compression effect by minimizing the probability weight path length. The following paragraphs describe how to code the quadtree to generate A, T, C, G data.

The proposed DNA encoding process [19]–[23] is derived from the Shannon coding and Fano coding. Assume a random set of information has the occurrence probability of a specific character i as p_i . We set the probability mass function (PMF) of random information as the probability vector

$$\mathbf{p} = (p_1, \dots, p_n) \quad (6)$$

We use $(s_i, \dots, s_r)$ integers to divide the probability vectors into four sets

$$s_i + \dots + s_r = 1 + \dots + n \quad (7)$$

Assume that the probability vectors in each set are combined as

$$(p_{s_i}, \dots, p_{s_r}), \quad 1 \leq i \leq r \leq n \quad (8)$$

In four sets, the total probability of each set can be recorded as q_t

$$q_t = p_{s_i} + \dots + p_{s_r}, \quad t = 1, \dots, 4 \quad (9)$$

We sort the character probabilities of the random information in descending order, so that $p_1 \geq p_2 \dots \geq p_n$. The set of probability vectors is then divided into four groups, where the probability sum of each group is as equal as possible, as indicated by the following equation.

$$\min \left(\frac{1}{4} \sum_{t=1}^4 \sum_{t'=1}^4 |q_t - q_{t'}| \right) \quad (10)$$

For a segmented collection, the codewords 'A', 'G', 'C', 'T' can be assigned in turn. After the code word is assigned, the quadrupling process continues until the number of random information characters in the set is smaller than four. The detailed steps to explain the algorithm are shown below.

- Design a valid coding table.
- For the coding table, the corresponding probability of each data is calculated.
- Sort the data in descending order according to probability to form a descending data set, so that the highest probability data is at the top of the quadruple tree and the lowest probability data is at the bottom of the quadruple tree.
- Segmenting the data set into four groups with nearly equal probabilities for each group.
- The four groups are assigned DNA bases A, G, C, T, respectively.

f) Step d and step e are recursively segmented, and each data set is subdivided and added with DNA bases A, G, C, T, until the data set is synthesized into indivisible data leaves.

We can push this encoding algorithm to time complexity. We let $S_1, \dots, S_m$ be source characters and $A_1, \dots, A_r$ be code alphabet [24], where $m = r + k(r-1)$ for $k \geq 0$ and $r \geq 4$. The number of partitionings can be defined by

$$\binom{\left(\frac{m-r}{r-1} + r - 1\right)}{r-1} = \frac{\left(\frac{m-r}{r-1} + r - 1\right)!}{\left(\frac{m-r}{r-1}\right)! (r-1)!} \quad (11)$$

If the loop is r steps, the total number of steps of encoding algorithm is

$$\begin{aligned} r \frac{(k+r-1)!}{k! (r-1)!} &= r \frac{(k+r-1)(k+r-2) \dots (k+1)k!}{k! (r-1)!} \\ &= \frac{A_0 + A_1 k + \dots + A_{r-2} k^{r-2} + A_{r-1} k^{r-1}}{(r-1)!} \end{aligned} \quad (12)$$

where $A_0 = r(r-1)!$, $A_{r-1} = r$, $A_i = f_i(r)$, and $f_i(r)$ is function of r . The formula can be written as

$$r + \frac{A_1}{(r-2)!} k + \dots + \frac{A_{r-2}}{(r-2)!} k^{r-2} + \frac{1}{(r-2)!} k^{r-1} \quad (13)$$

When $m = r + k(r-1)$ and $k = \frac{m-r}{r-1}$. The equation can be as

$$\begin{aligned} r + \frac{A_1}{(r-2)!} \frac{m-r}{r-1} + \dots + \frac{A_{r-2}}{(r-2)!} \left(\frac{m-r}{r-1}\right)^{r-2} \\ + \frac{1}{(r-2)!} \left(\frac{m-r}{r-1}\right)^{r-1} \end{aligned} \quad (14)$$

We expand the terms so the formula can be written as

$$B_0 + B_1 m + \dots + B_{r-2} m^{r-2} + B_{r-1} m^{r-1} \quad (15)$$

We can get time complexity of encoding algorithm. For fixed value of r , B_{r-1} is a constant and time complexity is $O(m^{r-1})$.

In Table I, we present an example to illustrate the encoding process. In particular, there are eight data types, marked as z_1 to z_8 in descending order of their probabilities, involved in the original source. The specific probability of each data type is shown in Table I. The encoding process splits the first data type into a data collection. The last two data sets (i.e. z_7 and z_8) are grouped into a single data collection. z_4 and z_5 form set of data. z_6 to z_8 form set of data. A, G, C, T are then allocated to each data set. The splitting process proceeds recursively if there are fewer than four types of aggregated data.

D. DNA Error Correction Scheme

The amplification and sequencing steps after DNA synthesis will result in insertion, deletion, replacement and reading errors. Therefore, choosing an appropriate error correction scheme is an important issue in DNA storage. Many current error correction solutions employ multiple gene sequence alignment algorithms. For example, the authors in [25], [26] use a mixed suffix tree method to perform multiple sequence

Algorithm 2 Quadtree Encoding

```

1: function Division(D)
2:   if D.size() < 4 then
3:     return ;
4:   end if
5:   divide D to D1, D2, D3, D4 with same probability.
6:   add A for D1.
7:   add G for D2.
8:   add C for D3.
9:   add T for D4.
10:  Division(D1)
11:  Division(D2)
12:  Division(D3)
13:  Division(D4)
14: end function
15:
16: function QEncoding(A, p, r, D)
17:  count data probability
18:  sort data probability with PSort(A, p, r)
19:  Division(D)
20:  output(DNA Data)
21: end function

```

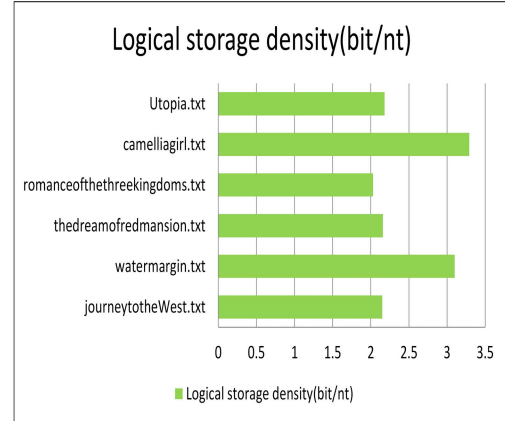
TABLE I

ASSUME A DATA TYPE OF SOURCE DATA z_1 TO z_8 , AND THEIR PROBABILITY DECREASES IN TURN. FOR THE FIRST SPLIT, THE FIRST DATA IS ASSIGNED CODEWORD A AS A DATA COLLECTION. TWO DATA ARE ASSIGNED G CODEWORDS AS A DATA COLLECTION. z_4 AND z_5 AS A DATA COLLECTION, ASSIGN CODEWORD C, z_6 TO z_8 ASSIGN CODEWORD T AS A DATA COLLECTION. THE SPLITTING PROCESS PROCEEDS RECURSIVELY UNTIL LESS THAN FOUR DATA ARE PRESENT IN THE DATA, RESULTING IN THE DNA ENCODING OF EACH DATA

Data	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8
Probability	0.25	0.14	0.12	0.12	0.11	0.09	0.09	0.08
First division	A	G		C		T		
Second division	A	A	G	A	G	A	G	C
Codewords	AA	GA	GG	CA	CG	TA	TG	TC

alignment. The disadvantage of the algorithm is that in low coverage cases, correct DNA base data can sometimes be turned into incorrect base data. Due to the high probability of errors in the sequencing process, it will be fatal when the amount of data is large. For the error correction scheme, this paper adopts the algorithm, RECKONER, proposed in the paper [12] to correct the read error. It mainly uses the quality index and counting method of base data to correct possible erroneous data.

In first step, the number of substrings of length k is calculated by counting k -mers in the input base data. The result of the next count is used to determine if the data needs to be read for correction. The validation of the threshold comes from the fact that an incorrect k -mer data occurs only once or twice. Because genome coverage may be caused by selecting a large number, the upper limit of the threshold is identified as 5. The error correction scheme achieves the

**Fig. 6.** Text compression logic density.

maximum probability of approaching the correct k -length data by correcting the possible wrong bases. $count(x)$ is number of k -mer. $weight(x)$ is 1 or 0.5, if x is included extending bases or not. $prob(i)$ is probability of i -th symbol changed. The region's correction rate is below

$$rate = \frac{(\sum_{x \in K^*} weight(x) count(x)) (\prod_{i=a}^b prob(i))}{\sum_{x \in K^*} weight(x)} \quad (16)$$

IV. EXPERIMENTS

In the experiments, we use source data as large text files and image files [27] (images also include others) to validate the performance of the proposed scheme. To demonstrate the end-to-end properties of the proposed method, DNA base data is generated. The size of the source data used in the experiment varies from several KB to MB. We mainly evaluate the performance of the proposed algorithm in terms of how much the data can be compressed and the image recovery quality can be improved when the source data is extremely large. To verify the effectiveness of the proposed method and its performance characteristics, we use compression efficiency and PSNR as the performance metrics. The encoding environment in the experiment is Ubuntu server system, i7 processor, 16GB memory hardware-based system encoding.

The source data of text compression includes both ancient and modern Chinese and foreign classics. There are data such as Chinese Journey to the West and foreign Tea flower girls. The logical storage density shown in Figure 6 indicates that all source data compressed text exceeds 2 bit/nt. Among them, the logical storage density of the source text data from the water margin exceeds 3bit/nt. This is because the proposed algorithm is designed based on the probability statistical model, and makes full use of the probability characteristics to remove redundancy. The structure of quadtree makes the probability data stored at the top of the tree and the small probability data stored at the bottom of the tree. Because of the characteristics of probability and tree, text data with large redundancy can be compressed better. The logical storage density of the proposed algorithm in text compression exceeds most of the state-of-the-art DNA storage algorithms, which

TABLE II
EXPERIMENTAL RESULTS OF DIFFERENT STORAGE SCHEMES

Study	Logical storage density (bits/nt)	Error correction	Full recovery	Random access
Wang [6]	1.67	No	No	No
Yeong [7]	3.37	No	No	No
Deng [8]	1.98	No	Yes	No
Erlich [9]	1.98	Yes	Yes	No
Goldman [28]	0.34	Yes	No	No
James [29]	0.02	No	No	Yes
Dimopoulou [30]	2.14	Yes	Yes	No
Proposed method	3.83	Yes	Yes	No

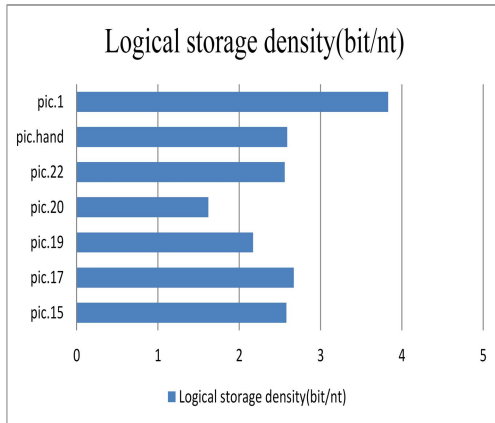


Fig. 7. Image compression logic density.

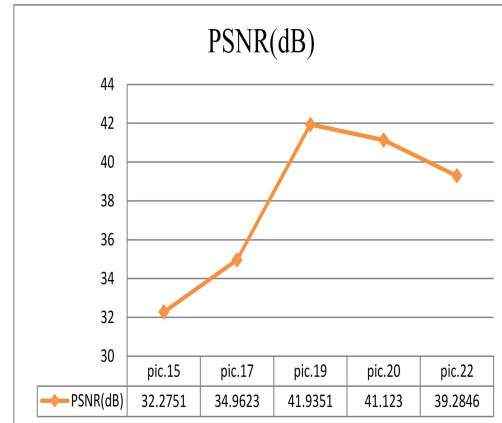


Fig. 8. PSNR of reconstruction image.

proves the superiority of the performance of the proposed algorithm.

The proposed algorithm also encodes images. It can be seen from Figure 7 that the logical density of most pictures exceeds 2 bit/nt after DNA encoding. The logical density of pic.1 of very large images (i.e. 250MB) reaches 3.83bit/nt. In addition, the influence of the proposed algorithm on logical density is due to the different number of partitions. If there are more types of source data, the quadtree will be divided more times and the tree depth will be higher, and the logical density of compression will increase. As the number of partitions increases, the superiority of quadtree algorithm performance increases.

At the same time, we also evaluate the visual quality after image reconstruction. Figure 8 shows the peak signal-to-noise ratio (PSNR) results of several reconstructed images. The PSNR of the images are all between 30dB and 42dB, which indicates that the quality of the reconstructed image is high after algorithm rebuilding.

Table II shows the experimental results of related DNA information storage schemes. Table shows that the proposed algorithm achieves higher logical storage density than existing works do. In addition, the proposed algorithm can convert the source data file to DNA base data end-to-end, store DNA directly, and have more operational space for subsequent biosynthesis.

Different read lengths are used in the error correction process in Table III. The random errors with probability of 1 / 20000, 1 / 10000, 1 / 5000 and 1 / 2500 are introduced to simulate different situations of actual errors. The corrected

TABLE III
RESULT OF ERROR CORRECTION

Read Length	Probability of error	Error Frequency after correct
150bp	1/20000	0
	1/10000	0
	1/5000	0
	1/2500	0.09%
100bp	1/20000	0
	1/10000	0
	1/5000	0.11%
	1/2500	0.12%
50bp	1/20000	0
	1/10000	0.16%
	1/5000	0.12%
	1/2500	0.07%

TABLE IV
EXTREME IMAGE CODING

colour percentage		(0,4.43%)	(4.43%-100%)
Method	Ours	Encode/Decode	Encode/Decode
	DNA code [11]	Decode Error	Encode/Decode

DNA data were compared with the uncorrected DNA data. Evaluate DNA data that cannot be recovered. Finally, the corrected error probability is obtained. DNA calibration data are shown in the Table III.

Table IV is the result of the extreme image encoded and decoded by the proposed algorithm over a small area of colors. Experiments show that the proposed algorithm can still encode and decode normally with a small proportion of color areas.

We use rate and psnr to describe the quality of image in Figure 9. We use our method and Goldman method to test

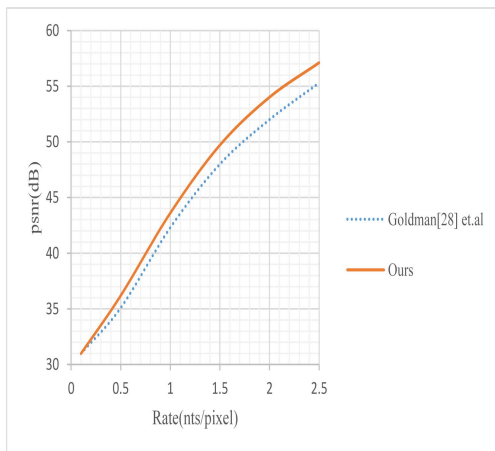


Fig. 9. Rate curve.

some image files. The results show that our code's reconstruction quality is slightly better than proposed method by Goldman *et al.* In the case of the same image reconstruction quality, the speed of our algorithm is smaller, and we can reconstruct better quality images.

V. CONCLUSION

In this work, a Quadtree end-to-end coding is proposed for DNA storage. The base data encoded by the quadtree coding method has been experimentally verified to obtain good results. This work is the first time that the source data files are directly encoded as DNA base data, which can significantly simplify the coding process. The encoding algorithm in this work cannot only convert the digital media information directly into DNA base data, but also compress the image into base data. Quadtree coding is superior to the state-of-the-art works in terms of logical storage density, and end-to-end generation of DNA bases, which makes subsequent biotechnology synthesis and other aspects have more operation space and reduce error generation.

Compression is an effective approach to reduce costs for subsequent biological studies, such as sequencing and synthesis. The excellent performance of this work will pave the foundation for the subsequent study of the entire DNA storage system. In the future research, we hope to introduce innovative biosynthetic and storage experiments to make the data more stable in DNA and achieve a complete solution of DNA storage.

REFERENCES

- [1] L. Ceze, J. Nivala, and K. Strauss, "Molecular digital data storage using DNA," *Nature Rev. Genet.*, vol. 20, no. 8, pp. 456–466, Aug. 2019.
- [2] L. Sun, J. He, J. Luo, and D. H. Coy, "DNA and the digital data storage," *Health Sci. J.*, vol. 13, no. 3, pp. 1–7, 2019.
- [3] Z. Ping *et al.*, "Carbon-based archiving: Current progress and future prospects of DNA-based data storage," *GigaScience*, vol. 8, no. 6, Jun. 2019, Art. no. giz075.
- [4] B. Cao, X. Zhang, J. Wu, B. Wang, Q. Zhang, and X. Wei, "Minimum free energy coding for DNA storage," *IEEE Trans. Nanobiosci.*, vol. 20, no. 2, pp. 212–222, Apr. 2021.
- [5] Y. Wang, M. Noor-A-Rahim, J. Zhang, E. Gunawan, Y. L. Guan, and C. L. Poh, "High capacity DNA data storage with variable-length oligonucleotides using repeat accumulate code and hybrid mapping," *J. Biol. Eng.*, vol. 13, no. 1, pp. 1–11, Dec. 2019.
- [6] Y. Wang, M. Noor-A-Rahim, E. Gunawan, Y. L. Guan, and C. L. Poh, "Construction of bio-constrained code for DNA data storage," *IEEE Commun. Lett.*, vol. 23, no. 6, pp. 963–966, Jun. 2019.
- [7] Y. Choi *et al.*, "High information capacity DNA-based data storage with augmented encoding characters using degenerate bases," *Sci. Rep.*, vol. 9, no. 1, pp. 1–7, Dec. 2019.
- [8] L. Deng *et al.*, "Optimized code design for constrained DNA data storage with asymmetric errors," *IEEE Access*, vol. 7, pp. 84107–84121, 2019.
- [9] Y. Erlich and D. Zielinski, "DNA Fountain enables a robust and efficient storage architecture," *Science*, vol. 355, no. 6328, pp. 950–954, 2017.
- [10] S. L. Shipman, J. Nivala, J. D. Macklis, and G. M. Church, "CRISPR–Cas encoding of a digital movie into the genomes of a population of living bacteria," *Nature*, vol. 547, no. 7663, pp. 345–349, 2017.
- [11] S. Zhang, B. Huang, X. Song, T. Zhang, H. Wang, and Y. Liu, "A high storage density strategy for digital information based on synthetic DNA," *3 Biotech*, vol. 9, no. 9, p. 342, Sep. 2019.
- [12] M. Długosz and S. Deorowicz, "RECKONER: Read error corrector based on KMC," *Bioinformatics*, vol. 33, no. 7, pp. 1086–1089, 2017.
- [13] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "Toward a DNA-based archival storage system," *IEEE Micro*, vol. 37, no. 3, pp. 98–104, Jun. 2017.
- [14] B. A. Olshausen and D. J. Field, "Natural image statistics and efficient coding," *Netw., Comput. Neural Syst.*, vol. 7, no. 2, pp. 333–339, 1996.
- [15] P. A. Abdulla *et al.*, "Efficient handling of string-number conversion," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Design Implement.*, Jun. 2020, pp. 943–957.
- [16] C. A. Hoare, "Quicksort," *Comput. J.*, vol. 5, no. 1, pp. 10–16, 1962.
- [17] Y. Yao, "Using nonlinear difference equations to study quicksort algorithms," *J. Difference Equ. Appl.*, vol. 26, no. 2, pp. 275–294, Feb. 2020.
- [18] A. Habib, M. J. Islam, and M. S. Rahman, "A dictionary-based text compression technique using quaternary code," *Iran J. Comput. Sci.*, vol. 3, no. 3, pp. 127–136, 2019.
- [19] C. E. Shannon, "A mathematical theory of communication," *ACM SIGMOBILE Mobile Comput. Commun. Rev.*, vol. 5, no. 1, pp. 3–55, 2001.
- [20] R. V. Hartley, "Transmission of information 1," *Bell Syst. Tech. J.*, vol. 7, no. 3, pp. 535–563, 1928.
- [21] D. Adjeroh and F. Nan, "Suffix-sorting via Shannon-Fano-Elias codes," *Algorithms*, vol. 3, no. 2, pp. 145–167, Apr. 2010.
- [22] T. Gaggie, "Dynamic Shannon coding," in *Proc. Eur. Symp. Algorithms*. Springer, 2004, pp. 359–370.
- [23] S. Krajci, C.-F. Liu, L. Mikes, and S. M. Moser, "Performance analysis of Fano coding," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2015, pp. 1746–1750.
- [24] L. Rueda and B. Oommen, "A fast and efficient nearly-optimal adaptive Fano coding scheme," *Inf. Sci.*, vol. 176, no. 12, pp. 1656–1683, Jun. 2006.
- [25] A. Allam, P. Kalnis, and V. Solovyev, "Karect: Accurate correction of substitution, insertion and deletion errors for next-generation sequencing data," *Bioinformatics*, vol. 31, no. 21, pp. 3421–3428, Nov. 2015.
- [26] S. Deorowicz, M. Kokot, S. Grabowski, and A. Debudaj-Grabysz, "KMC 2: Fast and resource-frugal k -mer counting," *Bioinformatics*, vol. 31, no. 10, pp. 1569–1576, May 2015.
- [27] R. Franzen, "Kodak lossless true color image suite," vol. 4, no. 2, 1999. [Online]. Available: <http://r0k.us/graphics/kodak>
- [28] N. Goldman *et al.*, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, pp. 77–80, Jan. 2013.
- [29] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "Toward a DNA-based archival storage system," *IEEE Micro*, 2017.
- [30] M. Dimopoulou, M. Antonini, P. Barbry, and R. Appuswamy, "A biologically constrained encoding solution for long-term storage of images onto synthetic DNA," in *Proc. 27th Eur. Signal Process. Conf. (EUSIPCO)*, Sep. 2019, pp. 1–5.